JapaLearn AI — Complete Technical & Product Documentation

Version: 1.0
Date: April 2026
Author: Tomide Williams (Founder)
Purpose: Comprehensive handoff guide for engineers, product teams, and future collaborators

---

Table of Contents

- Foreword — What Is JapaLearn AI?
- Chapter 1 — Product Vision & User Problem
- Chapter 2 — Tech Stack Overview
- Chapter 3 — Project Structure & File Map
- Chapter 4 — The User Journey (End to End)
- Chapter 5 — Frontend: Pages & Components
- Chapter 6 — The API Layer (All Endpoints)
- Chapter 7 — AI System: How We Generate Intelligence
- Chapter 8 — Database Schema (Supabase)
- Chapter 9 — Authentication & Security
- Chapter 10 — Information Quality & Accuracy Controls
- Chapter 11 — The Admin System
- Chapter 12 — Scoring & Assessment Logic
- Chapter 13 — The Curriculum & Learning Engine
- Chapter 14 — Design System & Styling
- Chapter 15 — Environment Variables & Configuration
- Chapter 16 — Known Weaknesses & Technical Debt
- Chapter 17 — Roadmap: From Validation to 100 Users and Beyond
- Appendix A — Full API Reference
- Appendix B — Glossary

Foreword — What Is JapaLearn AI? {#foreword}

JapaLearn AI is a web application built to help Nigerian professionals understand their chances of relocating abroad — and then actually prepare them to do it.

The word "Japa" is Nigerian slang for leaving the country. It is deeply cultural — millions of Nigerians want to relocate abroad for better opportunities, safety, education, and quality of life. The problem is the process is overwhelming, expensive to get wrong, and full of misinformation sold by unqualified agents.

JapaLearn AI solves this by:

1. Assessing a user's readiness based on their real profile (education, savings, language skills, experience)
2. Telling them which visa route makes the most sense for them

3. Building them a personalised learning curriculum to fill their gaps
4. Teaching them through AI-generated lessons grounded in real government policy sources
5. Tracking their progress as they work toward relocation

This documentation is the single source of truth for anyone who builds on, maintains, or hands off this codebase. It is written for engineers who may not know the product context, and also readable by non-technical founders who want to understand what they have built.

Chapter 1 — Product Vision & User Problem {#chapter-1}

1.1 The Problem We Solve

Relocating from Nigeria involves a maze of visa categories, eligibility thresholds, required documents, body recognitions, English tests, financial proofs, and country-specific rules. Most Nigerians attempting this process face three critical problems:

1. **Misinformation** — They rely on social media, WhatsApp groups, or paid agents who may be unqualified or incentivised to send them in the wrong direction.
2. **Wasted money** — They pursue visa routes they are not qualified for, losing application fees, preparation time, and emotional energy.
3. **No personalisation** — Generic immigration guides exist, but none adapt to a person's actual profile (their specialty, savings level, existing certifications, destination preference).

1.2 Who We Serve

JapaLearn AI is built for Nigerians across 8 professional categories:

Category	Examples
Tech Professionals	Software engineers, data scientists, cybersecurity, product managers
Medical Professionals	Medical doctors, nurses, pharmacists, physiotherapists
Skilled Workers	Accountants, civil engineers, architects, lawyers
Students	Undergraduate and postgraduate applicants
Business Owners	Entrepreneurs seeking business visas or investor routes
Freelancers	Remote workers, creatives, digital nomads
Family Reunification	Spouses and dependants of existing visa holders
Others	Anyone who doesn't fit the above categories

Supported destinations include: Canada, UK, USA, Germany, Ireland, Australia, Netherlands, Portugal, France, New Zealand, Sweden, Norway, UAE, Singapore, and others.

1.3 What Success Looks Like

The validation goal is the first 100 users completing a full cycle:

- Take quiz → Get report → Generate curriculum → Complete at least one lesson

Beyond validation, success means users reporting that JapaLearn AI gave them clarity, saved them money, and helped them reach a real visa outcome.

Chapter 2 — Tech Stack Overview {#chapter-2}

Every technical decision in this project was made with three constraints in mind: **speed to build, low cost at validation stage, and ability to scale later.**

2.1 Frontend

Tool	Version	Why We Use It
Next.js	16.2.2	Full-stack React framework. Handles both frontend pages and backend API routes in one codebase
React	19.2.4	UI library, powers all components
Tailwind CSS	3.4.19	Utility-first styling. Fast to build with, consistent design system
Framer Motion	12.38.0	Animations for page transitions, card entrances, interactive elements
Lucide React	1.7.0	Icon library, clean and consistent icon set
React Markdown	10.1.0	Renders AI-generated lesson content (which is in markdown format) to the browser

Next.js uses the **Pages Router** (not the newer App Router). This is intentional — the Pages Router is more stable and well-documented at the time of build.

2.2 Backend / API

Next.js API routes act as the backend. Every file inside `pages/api/` becomes a serverless API endpoint. No separate backend server is needed.

2.3 Database & Authentication

Tool	Purpose
Supabase	Managed PostgreSQL database + authentication + file storage
Supabase Auth	Handles email/password and Google OAuth login
pgvector	PostgreSQL extension for storing and searching AI embeddings (used for RAG)

Project ID: `sleohhuyxkgnvfbkhyke`
Region: `eu-west-1` (Ireland)

2.4 AI Models

JapaLearn AI uses a multi-model strategy. Different tasks use the best available model, with fallbacks in place:

Model	Provider	Used For
Gemini 1.5 Pro	Google	Readiness assessment, curriculum generation, lesson writing (primary)
GPT-4o-mini	OpenAI	Quiz generation, lesson summarisation, fallback for Gemini
Llama-3.3-70b-versatile	Groq	Configured as alternative, currently not primary
text-embedding-3-small	OpenAI	Generating vector embeddings for the RAG knowledge base

2.5 External APIs

API	Purpose
Google Custom Search API	Real-time search for current visa fees, timelines, policy changes
GOV.UK Content API	Structured extraction of official UK government policy pages
Supabase REST API	All database reads and writes

2.6 Hosting & Deployment

Currently: **Local development only.** The app is not yet deployed.

The intended deployment path is:

- **Vercel** (frontend + API routes) — connects to GitHub repo and auto-deploys on every push to `main`
- **Supabase** — already cloud-hosted

When deploying to Vercel, all environment variables (API keys, database URLs) must be added to the Vercel project settings.

Chapter 3 — Project Structure & File Map {#chapter-3}

```
japalearn-ai/
│
├── pages/                          # All pages and API routes
│   ├── _app.js                     # Global app wrapper (applies to all pages)
│   ├── index.js                    # Landing page (/)
│   ├── quiz.js                     # Readiness quiz (/quiz)
│   ├── report.js                   # AI report page (/report)
│   ├── login.js                    # Login page (/login)
│   ├── signup.js                   # Signup page (/signup)
│   ├── dashboard.js                # Main dashboard (/dashboard) — 66KB
│   ├── privacy.js                  # Privacy policy (/privacy)
│   ├── terms.js                    # Terms of service (/terms)
│   ├── blog.js                     # Blog listing (/blog)
│   ├── blog/[slug].js              # Blog post detail (/blog/uk-visa-guide etc)
│   ├── shared-profile.js           # Public referral profile
│   ├── u/[code].js                 # User public profile by referral code
```

```
│   ├── learn/
│   │   └── [curriculumId]/
│   │       └── [moduleIndex]/
│   │           ├── [lessonIndex].js  # Lesson viewer
│   │           └── quiz.js           # Module quiz
│   └── api/
│       ├── calculate-readiness.js  # POST: Score user + generate readiness report
│       ├── generate-curriculum.js  # POST: Build personalised learning curriculum
│       ├── generate-lesson.js      # POST: Create lesson content on demand
│       ├── generate-quiz.js        # POST: Create module quiz questions
│       ├── summarise-lesson.js     # POST: Return 6-point lesson summary
│       ├── test-ai.js              # POST: Dev endpoint to test AI
│       └── admin/
│           ├── add-document.js     # GET/POST/DELETE: Manage knowledge base
│           ├── ingest.js           # POST: Seed initial knowledge base
│           ├── research.js         # POST: Scrape and digest policy pages
│           ├── team.js             # GET/POST: Manage admin team
│           └── join.js             # GET/POST: Admin onboarding via invite
│
├── components/
│   ├── AuthCard.js                 # Login/signup form component
│   ├── layout/
│   │   ├── Navbar.jsx              # Top navigation bar
│   │   ├── Footer.jsx              # Footer
│   │   ├── Hero.jsx                # Landing hero section
│   │   ├── Stats.jsx               # Platform metrics display
│   │   ├── ProblemSection.jsx      # Pain points section
│   │   ├── Roadmap.jsx             # How JapaLearn works
│   │   ├── PathwaysMarquee.jsx     # Country/pathway animation strip
│   │   ├── Testimonials.jsx        # User testimonials section
│   │   ├── FAQ.jsx                 # Accordion FAQ section
│   │   └── NewsletterModal.jsx     # Email newsletter popup
│   ├── profile/
│   │   └── ReferralProfile.jsx     # Shared migration profile (demo data)
│   └── ui/
│       ├── button.jsx              # Reusable button component
│       └── progress.jsx            # Linear progress bar component
│
├── lib/
│   ├── ai.js                       # Master AI switchboard (model routing +
enrichment)
│   ├── supabase.js                 # Supabase client initialisation
│   ├── embeddings.js               # Text → vector embedding generation
│   ├── rag.js                      # RAG: store and retrieve knowledge base docs
│   ├── googleSearch.js             # Google Custom Search wrapper
│   ├── quizData.js                 # Quiz questions, segments, scoring maps
│   ├── skills.js                   # AI persona system prompts (SKILLS object)
│   ├── utils.js                    # Generic utility functions
│   └── Logo.js                     # Logo SVG component
│
├── constants/
│   └── blogData.js                 # Static blog post content and metadata
│
├── styles/
```

```
│   └── globals.css                # Global CSS, Tailwind directives, CSS
variables
│
├── public/
│   └── images/                    # Static images (logos, illustrations, etc.)
│
├── .env.local                     # Private environment variables (never commit
this)
├── next.config.js                 # Next.js configuration
├── tailwind.config.js             # Tailwind CSS configuration
├── jsconfig.json                  # JavaScript path aliases
└── package.json                   # Project dependencies and scripts
```

3.1 Key File Sizes and Complexity

File	Approximate Size	Complexity Note
pages/dashboard.js	66KB+	Most complex file. Multi-tab UI, all dashboard logic lives here
lib/skills.js	Large	Contains all AI persona prompts — the "brain" of the AI system
lib/quizData.js	Large	All quiz questions, segments, scoring rubrics
pages/api/calculate-readiness.js	Medium	Core scoring engine, calls AI + RAG + Google Search
pages/api/generate-curriculum.js	Medium	Curriculum builder, persona-specific enrichment

---

Chapter 4 — The User Journey (End to End) {#chapter-4}

This chapter traces the full path a user takes through JapaLearn AI, from first visit to completing a lesson. Understanding this flow is essential before touching any part of the codebase.

4.1 Journey Overview

```
Landing Page → Quiz → AI Report → Signup → Dashboard → Generate Curriculum → Take
Lesson → Take Module Quiz
```

4.2 Step 1 — Landing Page (`/`)

A user arrives at `pages/index.js`. They see:

- A hero section explaining the platform
- Platform stats (number of professionals helped, countries covered)
- A visual breakdown of how JapaLearn works (4-step roadmap)

- A scrolling marquee of pathways/countries
- Testimonials from users
- FAQ accordion
- A newsletter signup modal

Primary Call to Action: "Take the Quiz" button → redirects to `/quiz`

A user does not need to be logged in to take the quiz.

4.3 Step 2 — The Readiness Quiz (`/quiz`)

The quiz is the most important feature. It collects a user's profile across 3 phases:

Phase 1 — Professional Category: The user picks their category (Tech, Medical, Student, etc.)

Phase 2 — Segment: Based on their category, they pick a specific role (e.g., within "Medical", they pick "Medical Doctor" or "Nurse").

Phase 3 — Detailed Questions: Universal questions asked of everyone:

- Where do you want to relocate? (Destination country)
- What is your highest education level?
- How many years of professional experience?
- What is your English language test status? (IELTS/OET/Duolingo score or none taken)
- How much savings do you have? (In NGN ranges)
- What is your age range?
- What is your family situation? (Single, married, children)

Segment-specific questions (examples):

- Medical Doctor: Medical specialty, MDCN registration status, licensing exam progress (PLAB, USMLE, etc.)
- Software Engineer: Specialisation, certifications, existing job offer status
- Nurse: Nursing council registration, exam progress
- Student: Funding type (scholarship vs self-funded), field of study, sponsor details

Result: A JavaScript object called `answers` containing all responses. This is stored in browser `localStorage` and passed to the next step.

4.4 Step 3 — The AI Report (`/report`)

After the quiz, the user is sent to `pages/report.js`. The page immediately calls `/api/calculate-readiness` (POST) with the `answers` object.

While the AI processes (typically 5-15 seconds), the user sees a loading animation.

The report displays:

- A **Readiness Score** (0-100) shown as a donut chart
- **Candidate Status:** Strong (≥70), Moderate (40-69), Early Stage (<40)
- **Recommended Visa Route** for their destination and profile

- **Score Breakdown** across 5 dimensions: Financial, Language, Documentation, Professional Recognition, Process Knowledge
- **Top Strengths** and **Top Gaps** (coloured badge lists)
- **Estimated Cost** in Nigerian Naira
- **Estimated Timeline** (months)
- **Relocation Phases** (e.g., Phase 1: Document Preparation, Phase 2: Language Testing)
- **Feasibility Flag:** Green = realistic, Yellow = possible but challenging, Red = not feasible right now
- **4 Actionable Next Steps**
- A **CTA to Create an Account** to save the report and generate a curriculum

At this point, the AI data is also stored in `localStorage` as `ai_data`.

4.5 Step 4 — Signup or Login

From the report page, users are prompted to create an account. This takes them to `pages/signup.js` or `pages/login.js`.

Signup flow:

1. User enters full name, email, password
2. Or clicks "Continue with Google" (Google OAuth)
3. Supabase creates an auth user
4. The app reads `answers` and `ai_data` from localStorage
5. It writes a row to the `quiz_results` table linking the score and AI data to the user
6. User is redirected to `/dashboard`
7. Welcome message shows: "Welcome, [First Name]!"

Login flow:

1. Email/password or Google
2. Session established via Supabase JWT
3. Redirect to `/dashboard`
4. Welcome back message shows: "Welcome back, [First Name]!"

4.6 Step 5 — The Dashboard (`/dashboard`)

The dashboard is the user's home base. It has five tabs:

Tab	What It Shows
Overview	Score, visa route, strengths, gaps, recent activity
Learning	All generated curricula, module/lesson navigation
Roadmap	Phase-by-phase relocation timeline with costs
Resources	Document checklist, templates, country guides, cost calculators
Profile	Edit name/email/photo, dark mode toggle, retake quiz, referral code, delete account

On first login, the user has no curriculum yet. The Overview tab shows their score and a button to "Generate Your Curriculum."

4.7 Step 6 — Generate Curriculum

When the user clicks "Generate Curriculum," the dashboard calls `/api/generate-curriculum` (POST) with their `answers` and `ai_data`.

The AI builds a personalised learning curriculum structured as:

- 5-10 modules (each addressing a specific topic or gap)
- Each module has 2-5 lessons
- Modules are ordered so the biggest gap comes first
- The curriculum is saved to the `curricula` Supabase table

The Learning tab now shows the curriculum with modules and lessons listed.

4.8 Step 7 — Take a Lesson

When the user clicks a lesson, they are sent to `/learn/[curriculumId]/[moduleIndex]/[lessonIndex]`.

The lesson page calls `/api/generate-lesson` if the lesson hasn't been generated yet, or loads it from the `lesson_content` table if it exists.

Lessons contain:

- A lesson title and purpose statement
- Full markdown content (900-1200 words)
- 6 key takeaways (bullet points)
- An action step
- Source links (official government pages)
- An AI-generated summary option

When the user completes a lesson, it is marked in the `lesson_progress` table.

4.9 Step 8 — Module Quiz

After completing all lessons in a module, the user can take a 10-question quiz generated by `/api/generate-quiz`. This tests retention from the module's lessons.

Results are shown immediately with correct/incorrect feedback and explanations for each question.

Chapter 5 — Frontend: Pages & Components {#chapter-5}

5.1 Pages

`pages/_app.js`

The global app wrapper. Wraps all pages with any global providers or layout elements. Currently minimal — applies global styles via `styles/globals.css`.

`pages/index.js` — Landing Page

Assembles layout components in order: Navbar → Hero → Stats → ProblemSection → Roadmap → PathwaysMarquee → Testimonials → FAQ → Footer. Renders the NewsletterModal as an overlay.

State managed: Newsletter modal open/closed.

`pages/quiz.js` — Readiness Quiz

Manages a multi-phase quiz state machine:

```
phase: 'category' | 'segment' | 'questions' | 'submitting'
```

Internal state:

- `selectedCategory` — user's professional category
- `selectedSegment` — user's specific role
- `answers` — all question responses as a key/value object
- `currentQuestion` — index of active question
- `progress` — completion percentage

On completion: saves `answers` to `localStorage`, calls `/api/calculate-readiness`, saves `ai_data` to localStorage, redirects to `/report`.

`pages/report.js` — AI Report

Reads `answers` and `ai_data` from localStorage on mount. Renders the readiness report.

If no data exists (user navigated here directly), redirects to `/quiz`.

Uses a donut chart SVG for score visualisation. Uses colour-coded badges for strengths/gaps. Shows a feasibility flag banner (green/yellow/red).

`pages/login.js` — Login Page

Two-panel layout:

- Left panel: Product benefits and feature highlights
- Right panel: Login form (email + password) with Google OAuth option

On successful login: checks for pending quiz data in localStorage, saves it to DB if found, redirects to `/dashboard`.

`pages/signup.js` — Signup Page

Same two-panel structure as login. Adds full name field and password strength indicator.

On successful signup: creates user profile row in `profiles` table, saves quiz data to `quiz_results` table, redirects to `/dashboard`.

`pages/dashboard.js` — Main Dashboard

This is the largest and most complex file in the codebase (~66KB). It contains:

- All five tab views (Overview, Learning, Roadmap, Resources, Profile)
- Sidebar navigation (desktop: 260px fixed, mobile: overlay)
- Bottom navigation bar (mobile only)
- Global search with category filters
- Profile picture upload
- Curriculum and lesson management
- Dark mode logic
- Referral code generation

Key state variables:

```
activeTab          // Which main tab is visible
sidebarOpen        // Mobile sidebar toggle
searchQuery        // Global search input
curricula          // User's list of curricula from DB
selectedCurriculum // Currently active curriculum
lessonProgress     // Map of completed lessons
quizResults        // User's readiness score + ai_data
darkMode           // Boolean, persisted in localStorage
```

Data loading on mount:

1. Checks for Supabase session → redirects to `/login` if none
2. Fetches user profile from `profiles` table
3. Fetches most recent quiz result from `quiz_results` table
4. Fetches all curricula from `curricula` table
5. Fetches all lesson progress from `lesson_progress` table

`pages/learn/[curriculumId]/[moduleIndex]/[lessonIndex].js` — Lesson Viewer

Dynamic route. Extracts `curriculumId`, `moduleIndex`, `lessonIndex` from URL.

On mount:

1. Loads curriculum from DB
2. Checks if lesson content exists in `lesson_content` table
3. If not: calls `/api/generate-lesson` → saves result to DB → renders it
4. If yes: renders cached content

Shows: lesson content (rendered markdown), key takeaways accordion, action step, sources list, summary toggle.

On completion: writes to `lesson_progress` table.

`pages/learn/[curriculumId]/[moduleIndex]/quiz.js` — Module Quiz

Loads quiz data for the module. If no quiz exists, calls `/api/generate-quiz`. Presents 10 questions one by one. Shows results with score and explanation for each answer.

`pages/blog.js` — Blog Listing

Reads from `constants/blogData.js`. Shows a featured post, search bar, category filter tabs, and a grid of post cards.

`pages/blog/[slug].js` — Blog Post

Renders a full blog post with hero image, metadata (author, date, read time), markdown content, FAQ accordion, and related posts.

`pages/privacy.js` and `pages/terms.js`

Static legal pages. Important for NDPR compliance.

5.2 Layout Components

`components/layout/Navbar.jsx`

Top navigation bar. Logo on the left, links in the middle (Home, Quiz, Blog), auth buttons on the right (Login, Get Started). Responsive — collapses to hamburger on mobile.

`components/layout/Hero.jsx`

Landing page hero with headline, subtext, CTA buttons ("Take the Quiz", "See How It Works"), and a platform illustration or stats preview.

`components/layout/Stats.jsx`

Grid of metric cards. Currently shows: number of professionals helped, countries covered, visa routes available, quiz completions.

Note: These numbers are marketing claims. They should be updated to reflect real data as the platform grows.

`components/layout/FAQ.jsx`

Accordion-style FAQ. Content is hard-coded. Questions and answers cover what JapaLearn AI is, how the quiz works, whether it guarantees visa approval (it does not), pricing, and data privacy.

`components/layout/NewsletterModal.jsx`

Email capture modal. Appears after the user has been on the landing page for a set time. Submits email to a newsletter list (implementation detail may vary — check if actually connected to a provider).

5.3 Auth Component

`components/AuthCard.js`

Reusable authentication component used by both login.js and signup.js. Handles form state, validation errors, Google OAuth trigger, and form submission.

Validates:

- Email format
- Password length (minimum 8 characters)
- Full name presence (on signup)

5.4 UI Components

`components/ui/button.jsx`

Configurable button with variants (primary, secondary, outline, ghost) and sizes (sm, md, lg). Built with Radix UI's Slot component for composability.

`components/ui/progress.jsx`

Linear progress bar. Used in the dashboard to show lesson completion per module.

Chapter 6 — The API Layer (All Endpoints) {#chapter-6}

All API routes live in `pages/api/`. Each file is a serverless function. They receive an HTTP request and return JSON.

6.1 `POST /api/calculate-readiness`

What it does: The core scoring engine. Takes a user's quiz answers and returns a comprehensive migration readiness assessment.

Caller: `pages/quiz.js` (immediately after quiz completion)

Request body:

```
{
  "answers": {
    "category": "Tech",
    "segment": "Software Engineer",
    "destination": "UK",
    "education": "Bachelor's degree",
    "experience": "3-5 years",
    "language": "IELTS 6.5",
    "savings": "₦5M - ₦10M",
    "age": "25-30",
    "familySituation": "Single",
    "specialisation": "Backend Development",
    "certifications": "AWS Certified Solutions Architect",
    "jobOffer": "No"
```

```
  }
}
```

Process:

1. Enriches the prompt with RAG context (relevant policy documents from knowledge base)
2. Enriches with Google Search results (current fees, timelines, policy news)
3. Sends enriched prompt to Gemini 1.5 Pro (with OpenAI GPT-4o-mini as fallback)
4. Uses SKILLS.READINESS_SCORER persona (strict JSON output format)
5. Parses and returns the AI response

Response:

```
{
  "overall": 68,
  "flag": "yellow",
  "feasibilityStatus": "Realistic with effort",
  "topGaps": ["Language Skills", "Financial Runway"],
  "topStrengths": ["Work Experience", "Education Level", "Age"],
  "expertNote": "You have solid foundations for the UK Skilled Worker route, but
your IELTS score is below the B2 threshold required by most sponsors. This is your
most critical blocker.",
  "recommendedRoute": "UK Skilled Worker Visa",
  "destination": "UK",
  "dimensions": {
    "financial": 55,
    "language": 60,
    "documentation": 75,
    "professional": 80,
    "knowledge": 45
  },
  "estimatedCost": "₦6M – ₦12M",
  "estimatedTimelineMonths": 14,
  "phases": [
    {
      "name": "Language Preparation",
      "duration": "2-3 months",
      "actions": ["Enroll in IELTS prep course", "Take IELTS Academic test",
"Target Band 7+ overall"]
    }
  ],
  "nextSteps": [
    "Retake IELTS with a target of Band 7 or above",
    "Identify UK companies on the sponsor register in your specialisation",
    "Open a UK-linked savings account and build financial evidence",
    "Research the points calculator at UKVI website"
  ],
  "mismatchFound": false
}
```

Error handling: If AI fails to return valid JSON, the endpoint falls back to a static error response with a message asking the user to retry.

6.2 `POST /api/generate-curriculum`

What it does: Builds a personalised multi-module learning curriculum based on the user's profile and readiness assessment.

Caller: `pages/dashboard.js` (when user clicks "Generate Curriculum")

Request body:

```
{
  "answers": { /* full quiz answers object */ },
  "ai_data": { /* readiness assessment result */ }
}
```

Process:

1. Identifies the user's persona (Tech Professional, Medical Doctor, etc.)
2. Identifies the biggest gaps from `ai_data.topGaps`
3. Generates 4 persona-specific Google Search queries to gather current data
4. Performs those 4 searches
5. Retrieves RAG context for the persona
6. Sends all enriched context to Gemini with SKILLS.CURRICULUM_BUILDER persona
7. Validates the output is proper JSON
8. Returns the curriculum

Response:

```
{
  "curriculum": {
    "persona": "Software Engineer",
    "goal": "Secure a UK Skilled Worker Visa as a backend developer",
    "route": "UK Skilled Worker Visa",
    "readiness_level": "intermediate",
    "curriculum_title": "From Nigerian Dev to UK Tech Worker",
    "modules": [
      {
        "module_id": "m1",
        "title": "Cracking the UK Tech Job Market",
        "purpose": "Understand which roles sponsor visas and how to position
yourself",
        "priority": 1,
        "lessons": [
          {
            "title": "How UK Visa Sponsorship Works for Tech Roles",
            "goal": "Understand the Certificate of Sponsorship process",
```

```
              "difficulty": "beginner",
              "estimated_time_minutes": 20
            },
            {
              "title": "Building a CV That Gets UK Tech Interviews",
              "goal": "Adapt your CV to UK standards and expectations",
              "difficulty": "intermediate",
              "estimated_time_minutes": 25
            }
          ]
        }
      ],
      "gaps": ["Job search strategy", "UK salary expectations"],
      "next_best_action": "Research 10 UK companies on the sponsor register in your
tech stack"
    }
  }
```

Saved to: `curricula` table in Supabase, linked to the user's `user_id`.

6.3 `POST /api/generate-lesson`

What it does: Generates full lesson content for a specific lesson in a curriculum.

Caller: `pages/learn/[curriculumId]/[moduleIndex]/[lessonIndex].js`

Request body:

```
{
  "lessonTitle": "How UK Visa Sponsorship Works for Tech Roles",
  "moduleTitle": "Cracking the UK Tech Job Market",
  "curriculumTitle": "From Nigerian Dev to UK Tech Worker",
  "destination": "UK",
  "segment": "Software Engineer",
  "curriculumId": "abc123",
  "moduleIndex": 0,
  "lessonIndex": 0
}
```

Process:

1. First checks if lesson already exists in `lesson_content` table → returns cached version if found
2. If not: retrieves RAG context + Google Search results for the lesson topic
3. Sends to AI with SKILLS.LESSON_WRITER persona
4. AI generates 900-1200 word lesson in markdown
5. Second AI call extracts: key_takeaways (6 bullet points), action_step, sources
6. Saves complete lesson to `lesson_content` table
7. Returns lesson object

Response:

```
{
  "lesson": {
    "title": "How UK Visa Sponsorship Works for Tech Roles",
    "content": "# How UK Visa Sponsorship Works for Tech Roles\n\n## Why This
Matters\n...",
    "key_takeaways": [
      "UK employers must hold a Sponsor Licence from the Home Office before they
can hire a non-UK worker — this is non-negotiable.",
      "The Certificate of Sponsorship (CoS) is a unique alphanumeric code issued
by your employer, not a physical document...",
      "..."
    ],
    "action_step": "Go to the UK government sponsor register and search for
companies in your tech stack that are actively sponsoring.",
    "sources": [
      {
        "label": "UK Home Office — Skilled Worker Visa",
        "url": "https://www.gov.uk/skilled-worker-visa"
      }
    ]
  }
}
```

6.4 `POST /api/generate-quiz`

What it does: Generates 10 multiple-choice quiz questions for a specific module.

Caller: `pages/learn/[curriculumId]/[moduleIndex]/quiz.js`

Request body:

```
{
  "curriculumId": "abc123",
  "moduleIndex": 0
}
```

Process:

1. Loads the curriculum from `curricula` table to get the module and lesson titles
2. Sends module context to OpenAI GPT-4o-mini in JSON mode
3. Instructs AI to generate questions across all lessons in the module
4. Returns 10 questions with 4 options each, correct answer index, and explanation

Response:

```
{
  "quiz": {
    "moduleTitle": "Cracking the UK Tech Job Market",
    "questions": [
      {
        "id": 1,
        "question": "Which document does a UK employer provide to allow a skilled
worker to apply for a visa?",
        "options": [
          "A work permit",
          "A Certificate of Sponsorship (CoS)",
          "A letter of employment",
          "An entry clearance vignette"
        ],
        "correctIndex": 1,
        "explanation": "The Certificate of Sponsorship is a unique reference
number issued by a licensed UK employer. It is not a physical certificate but an
electronic record on the Home Office system."
      }
    ]
  }
}
```

6.5 `POST /api/summarise-lesson`

What it does: Takes a full lesson's markdown content and returns a 6-point summary.

Caller: Lesson viewer page (when user clicks "Summarise This Lesson")

Request body:

```
{
  "content": "# Full lesson content in markdown...",
  "title": "How UK Visa Sponsorship Works for Tech Roles"
}
```

Response:

```
{
  "points": [
    "UK employers need a Sponsor Licence before they can hire overseas workers —
without it, they legally cannot offer you a sponsored role.",
    "...",
    "..."
  ]
}
```

6.6 `POST /api/test-ai`

What it does: Developer endpoint to verify AI connectivity. Not used in production features.

Request:

```
{ "message": "Hello, what can you do?" }
```

Response:

```
{
  "success": true,
  "provider": "Gemini 1.5 Pro",
  "response": "I can help with..."
}
```

6.7 Admin API Endpoints

These endpoints are protected. Only accessible to users with admin credentials (see Chapter 11).

`GET|POST|DELETE /api/admin/add-document`

Manages the RAG knowledge base.

- `GET`: Returns all documents in `migration_documents` table
- `POST`: Adds a new policy document. Auto-generates embedding. Body: `{ content, country, category, source_url }`
- `DELETE`: Removes a document by ID. Query: `?id=document_id`

`POST /api/admin/ingest`

Seeds the database with an initial set of foundational migration documents. Run once during setup. Not idempotent — running twice will create duplicates.

`POST /api/admin/research`

Scrapes and digests an official policy URL into a structured document draft.

Request:

```
{ "url": "https://www.gov.uk/skilled-worker-visa" }
```

Process:

1. Detects if URL is a GOV.UK URL → uses GOV.UK Content API for clean extraction
2. Otherwise: fetches and strips HTML to plain text
3. Sends extracted content to AI for distillation into a structured markdown guide
4. Returns a draft for admin review (does not auto-save)

Response:

```
{
  "success": true,
  "draft": {
    "content": "# Skilled Worker Visa — UK\n\n## Overview\n...",
    "country": "UK",
    "category": "visa",
    "source_url": "https://www.gov.uk/skilled-worker-visa"
  }
}
```

Admin then decides to save or discard the draft via the `add-document` endpoint.

`GET|POST /api/admin/team`

Manages the admin team.

- `GET`: Returns list of approved admins and pending invites
- `POST` with `action: "invite"`: Sends invite to a new email
- `POST` with `action: "approve"`: Approves a pending admin
- `POST` with `action: "reject"`: Rejects a pending admin
- `POST` with `action: "remove"`: Revokes admin access

`GET|POST /api/admin/join`

Handles the invite acceptance flow for new admins.

- `GET`: Validates an invite token (from email link)
- `POST`: Creates the admin user from the invite

Chapter 7 — AI System: How We Generate Intelligence {#chapter-7}

This chapter explains how JapaLearn AI's intelligence works under the hood. This is critical to understand before changing anything related to AI responses.

7.1 The Master AI Switchboard (`lib/ai.js`)

All AI calls in the codebase go through `lib/ai.js`. This is the single gateway. It does two things:

1. **Enriches** the prompt with real-world context (if requested)
2. **Routes** to the right AI model

```
// Simplified flow
async function generateResponse(messages, skillPrompt, options) {
  if (options.enrich) {
    const ragContext = await retrieveContext(query)
    const searchResults = await googleSearch(query)
    messages = mergeContext(messages, ragContext, searchResults)
  }

  if (GEMINI_API_KEY) {
    return callGemini(messages, skillPrompt)
  } else {
    return callOpenAI(messages, skillPrompt)
  }
}
```

7.2 The Enrichment Layer

Enrichment is what makes JapaLearn AI answers grounded rather than hallucinated.

When `options.enrich = true` is passed:

Step 1 — RAG (Retrieval-Augmented Generation): The system searches the `migration_documents` table using vector similarity. It converts the query to an embedding (1536-dimensional vector) and finds the most semantically similar documents. Documents with similarity ≥ 0.7 are included as context.

This gives the AI access to curated, verified policy documents we've manually ingested and maintained.

Step 2 — Google Custom Search: The system performs a targeted Google search (using our custom search engine) and returns the top 5 results as formatted snippets. This catches recent changes — new fees, updated processing times, policy announcements — that our static knowledge base might not have.

Combined context is injected into the AI prompt as:

```
=== VERIFIED POLICY CONTEXT (from our knowledge base) ===
[RAG results here]

=== CURRENT SEARCH RESULTS (live web) ===
[Google Search snippets here]
```

This two-layer approach means: RAG handles depth (detailed policy knowledge), Search handles freshness (current news and changes).

7.3 The Skills System (`lib/skills.js`)

Rather than having generic prompts, JapaLearn AI uses a **persona system** called SKILLS. Each skill is a carefully engineered system prompt that defines:

- Who the AI is pretending to be

- What its mission is for this specific task
- What format it must output (JSON structure, word count, etc.)
- What it must NOT do (hallucinate fees, use vague language, etc.)

Available Skills:

`SKILLS.READINESS_SCORER`

- **Persona:** Senior migration analyst with 15 years of Nigerian applicant experience
- **Mission:** Assess feasibility, score across 5 dimensions, identify killer criteria
- **Key rules:**
  - Savings below ₦5M for non-scholarship routes = flag red immediately
  - Score gaps honestly even if it hurts
  - Recommend based on actual visa requirements, not what the user wants to hear
  - Output strict JSON (no prose, no commentary outside the JSON)
- **Output format:** JSON matching the response schema shown in Chapter 6.1

`SKILLS.CURRICULUM_BUILDER`

- **Persona:** Nigerian migration education specialist
- **Mission:** Design a personalised learning journey that addresses actual gaps
- **Key rules:**
  - Biggest gap = Module 1 (blockers first)
  - 5-10 modules maximum
  - 2-5 lessons per module
  - No placeholder content ("Module X: TBD")
  - Use real exam names, body names, fee amounts
  - Order must be logical — early modules unlock later ones
- **Output format:** Strict JSON curriculum structure

`SKILLS.LESSON_WRITER`

- **Persona:** Nigerian migration educator and policy expert
- **Mission:** Write a complete lesson grounded in official sources
- **Key rules:**
  - 900-1200 words
  - 9-part lesson structure (title → purpose → why it matters → prerequisites → explanation → step-by-step → common mistakes → self-check → next action)
  - Ground every fact in RAG or Search context
  - Include Nigerian-specific costs (₦ amounts) and context (NYSC, NIN, etc.)
  - 6 key takeaways minimum
  - Must include source URLs
- **Output format:** Markdown content + structured metadata

`SKILLS.QA_ASSISTANT`

- **Persona:** JapaLearn expert assistant
- **Mission:** Answer user questions about migration

- **Key rules:**
  - Use RAG first, Search for freshness
  - Say "I don't have current information on this" rather than hallucinating
  - Direct users to official sources

7.4 Model Selection Logic

```
If GEMINI_API_KEY is set → Use Gemini 1.5 Pro (primary)
If Gemini fails → Fall back to OpenAI GPT-4o-mini

For quiz generation → Always use OpenAI (JSON mode reliability)
For lesson summarisation → Always use OpenAI (speed + cost)
```

The reason Groq / Llama is configured but not primary: Groq is fast and free, but the Llama model produced less reliable structured JSON outputs during testing. It remains available as a fallback option if API costs need to be reduced.

7.5 RAG Deep Dive (`lib/rag.js`)

RAG stands for Retrieval-Augmented Generation. It is the system that allows the AI to access our curated knowledge base.

How documents are stored:

```
async function addDocument(content, metadata) {
  const embedding = await generateEmbedding(content)  // 1536-dim vector
  await supabase.from('migration_documents').insert({
    content,
    embedding,
    country: metadata.country,
    category: metadata.category,
    source_url: metadata.source_url
  })
}
```

How documents are retrieved:

```
async function retrieveContext(query) {
  const queryEmbedding = await generateEmbedding(query)
  const { data } = await supabase.rpc('match_documents', {
    query_embedding: queryEmbedding,
    match_threshold: 0.7,
    match_count: 5
  })
  return data.map(d => d.content).join('\n\n')
}
```

The `match_documents` function is a PostgreSQL function using pgvector's cosine similarity search.

Current knowledge base includes:

- UK Skilled Worker Visa
- Canada Express Entry system
- Additional visa routes added via the admin research tool

Gap: The knowledge base is thin. As of this documentation, only a handful of documents have been ingested. Expanding this is one of the highest-value improvements available.

Chapter 8 — Database Schema (Supabase) {#chapter-8}

JapaLearn AI uses a managed PostgreSQL database via Supabase. All tables live in the `public` schema unless noted.

8.1 `auth.users` (Supabase Built-In)

Managed entirely by Supabase Auth. We do not write to this table directly.

Column	Type	Description
id	UUID	Primary key, referenced by all other tables as `user_id`
email	text	User email address
created_at	timestamp	Account creation time
email_confirmed_at	timestamp	When email was verified (null if unverified)

8.2 `profiles`

Stores additional user data beyond what Supabase Auth provides.

Column	Type	Description
id	UUID	Foreign key → auth.users.id
full_name	text	User's full name
profile_picture_url	text	URL to uploaded profile image (Supabase Storage)
created_at	timestamp	Row creation time
updated_at	timestamp	Last update time

8.3 `quiz_results`

Stores each quiz attempt.

Column	Type	Description
id	UUID	Primary key

Column	Type	Description
user_id	UUID	Foreign key → auth.users.id
answers	JSONB	Full quiz answers object
score	integer	Readiness score 0-100
ai_data	JSONB	Full readiness assessment result from AI
created_at	timestamp	When quiz was taken

A user can have multiple rows (retakes). The dashboard uses the most recent.

8.4 `curricula`

Stores generated curricula.

Column	Type	Description
id	UUID	Primary key (also used in lesson URLs)
user_id	UUID	Foreign key → auth.users.id
title	text	Curriculum title
destination	text	Target country
segment	text	User's professional segment
category	text	User's professional category
modules	JSONB	Full curriculum structure (modules + lessons array)
created_at	timestamp	Creation time
updated_at	timestamp	Last modification time

8.5 `lesson_content`

Stores generated lesson content (cached after first generation).

Column	Type	Description
id	UUID	Primary key
curriculum_id	UUID	Foreign key → curricula.id
module_index	integer	Zero-based module position
lesson_index	integer	Zero-based lesson position within module
title	text	Lesson title
content	text	Full lesson in markdown
key_takeaways	JSONB	Array of 6 takeaway strings

Column	Type	Description
action_step	text	Single actionable instruction
sources	JSONB	Array of {label, url} objects
created_at	timestamp	Generation time

Important: Lessons are generated on-demand and cached. The second time a user visits a lesson, it loads from this table, not from the AI. This saves API costs and ensures consistency.

8.6 `lesson_progress`

Tracks which lessons each user has completed.

Column	Type	Description
id	UUID	Primary key
user_id	UUID	Foreign key → auth.users.id
curriculum_id	UUID	Foreign key → curricula.id
module_index	integer	Module position
lesson_index	integer	Lesson position
completed	boolean	Whether lesson is marked complete
completed_at	timestamp	When it was completed (null if not complete)
created_at	timestamp	Row creation time

Unique constraint: `(user_id, curriculum_id, module_index, lesson_index)` — prevents duplicate rows.

8.7 `migration_documents`

The RAG knowledge base.

Column	Type	Description
id	UUID	Primary key
content	text	Full document text
embedding	vector(1536)	OpenAI embedding for similarity search
metadata	JSONB	Additional structured metadata
country	text	e.g., "UK", "Canada"
category	text	e.g., "visa", "education", "finance"
source_url	text	Where the content was sourced from
last_updated	date	When the source was last verified

Column	Type	Description
created_at	timestamp	Row creation time

8.8 `admin_users`

Controls who has admin access.

Column	Type	Description
id	UUID	Primary key
email	text	Admin user's email
full_name	text	Admin's full name
role	text	"admin" or "super_admin"
status	text	"pending", "approved", or "rejected"
created_at	timestamp	When they were added

8.9 `admin_invites`

Tracks invite tokens sent to new admins.

Column	Type	Description
id	UUID	Primary key
email	text	Invitee's email
role	text	Role being assigned
token	text	Unique invite token (in email link)
invited_by	text	Email of admin who sent the invite
used_at	timestamp	When the token was used (null if unused)
used_by	UUID	User ID who used it
created_at	timestamp	When the invite was created

Chapter 9 — Authentication & Security {#chapter-9}

9.1 Supabase Authentication

JapaLearn AI uses Supabase Auth for all user authentication. This handles:

- Password hashing (bcrypt, never stored in plaintext)
- Email verification
- JWT session tokens
- OAuth provider integration

The Supabase client is initialised in `lib/supabase.js`:

```
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

The `ANON_KEY` is safe to expose in the browser — it is scoped by Row Level Security (RLS) policies. The `SERVICE_ROLE_KEY` is server-only and bypasses RLS — it is only used in API routes.

9.2 Email/Password Flow

1. User submits signup form
2. `supabase.auth.signUp({ email, password, options: { data: { full_name } } })` is called
3. Supabase creates the auth user and sends a confirmation email
4. On confirm, user can log in
5. `supabase.auth.signInWithPassword({ email, password })` establishes a session

Session is stored in a browser cookie by Supabase. On every page load, `supabase.auth.getSession()` retrieves the current session.

9.3 Google OAuth Flow

1. User clicks "Continue with Google"
2. `supabase.auth.signInWithOAuth({ provider: 'google' })` is called
3. User is redirected to Google's consent screen
4. Google redirects back to the app's callback URL
5. Supabase handles the token exchange
6. Session is established

For Google OAuth to work in production, the callback URL must be added to both:

- Supabase Auth > URL Configuration
- Google Cloud Console > OAuth 2.0 credentials

9.4 Admin Authentication

Admin endpoints use a two-check system:

```
// In every admin API route:
const session = await getSession(req)

const isSuperAdmin = session.user.email === 'jappalearn@gmail.com'

if (!isSuperAdmin) {
  const { data: adminUser } = await supabase
    .from('admin_users')
```

```
      .select('status')
      .eq('email', session.user.email)
      .single()

    if (!adminUser || adminUser.status !== 'approved') {
      return res.status(403).json({ error: 'Not authorized' })
    }
  }
```

The super admin email (`jappalearn@gmail.com`) bypasses the database check. This is the founder's permanent override.

9.5 Row Level Security (RLS)

Supabase RLS policies ensure users can only access their own data. The key policies are:

- `quiz_results`: Users can only read/write rows where `user_id = auth.uid()`
- `curricula`: Users can only read/write rows where `user_id = auth.uid()`
- `lesson_progress`: Users can only read/write rows where `user_id = auth.uid()`
- `lesson_content`: Readable by all authenticated users (not user-scoped — lessons are shared cache)
- `migration_documents`: Readable by authenticated users, writable only via service role (API routes)

Important: RLS must remain enabled on all tables. Disabling RLS would expose all user data to any authenticated user.

9.6 Environment Variable Security

- `NEXT_PUBLIC_*` variables are embedded in the browser bundle — keep only non-sensitive values here
- `SUPABASE_SERVICE_ROLE_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`, `GROQ_API_KEY` are server-only — never expose these to the browser
- The `.env.local` file must never be committed to git (it is in `.gitignore`)

Chapter 10 — Information Quality & Accuracy Controls {#chapter-10}

This is one of the most important chapters in this document. JapaLearn AI gives people advice about major life decisions. Getting it wrong has real consequences.

Here is every mechanism we have in place to ensure accuracy, and where the gaps still are.

10.1 The Accuracy Stack

We use a layered approach — each layer adds a check or constraint on the AI's output:

Layer 1 — Curated Knowledge Base (RAG) Official policy documents from government sources are manually ingested, chunked, embedded, and stored in our database. When the AI answers a question, it first retrieves the most relevant policy context from this database.

This means the AI is not just generating from its training data — it is grounded in documents we have verified.

Current documents ingested:

- UK Skilled Worker Visa (GOV.UK)
- Canada Express Entry (IRCC)
- A small number of other visa routes

Gap: The knowledge base is sparse. Many countries and visa categories have no coverage. The AI falls back to its training knowledge, which may be outdated.

Layer 2 — Live Google Search Before every major AI response (scoring, curriculum, lessons), we perform a real-time Google Search. This catches:

- New processing fees
- Updated salary thresholds
- Recent policy changes
- New application processes

Our custom search engine (`CX: c74f946a24f824428`) is configured to prioritise authoritative sources like gov.uk, canada.ca, uscis.gov, etc.

Layer 3 — Strict AI Persona Constraints The SKILLS system prompts include explicit "never do" rules:

- Never invent visa fees — use real figures or admit you do not know
- Never guarantee visa approval
- Flag when information may be outdated
- The READINESS_SCORER is told: if savings are below ₦5M for non-scholarship routes, flag immediately regardless of other scores (this is a known killer criterion)

Layer 4 — Source Citation Requirement Every lesson must include source URLs. The LESSON_WRITER skill is instructed to cite official government pages for every major claim. Users can click through to verify.

Layer 5 — Feasibility Flags The scoring system uses three flags:

- Green (≥70): Realistic candidate — proceed with confidence
- Yellow (40-69): Possible but gaps need addressing — caution advised
- Red (<40): Not feasible right now — significant work needed first

These flags are shown prominently on the report and colour-code the entire experience.

Layer 6 — Legal Disclaimer The Terms of Service explicitly states: JapaLearn AI is an educational platform, not a visa agency or immigration law firm. The information provided is for guidance only and should not replace professional legal advice.

10.2 What Can Still Go Wrong

Known accuracy risks:

1. **Thin knowledge base.** If a user asks about a visa route we have not ingested, the AI answers from its training data. GPT-4o-mini's training cutoff and Gemini's training cutoff mean some information may be 1-2 years outdated.

2. **Google Search relevance.** Our search engine may surface SEO-optimised immigration agency blogs rather than official sources. The search is not restricted to .gov domains only.

3. **AI hallucination on edge cases.** Despite the constraints, language models can still produce plausible-sounding but incorrect numbers, especially for less common visa routes.
4. **Dynamic policy changes.** Visa rules change frequently (salary thresholds, fee increases, quota changes). Even fresh search results may lag behind changes made in the last few days.
5. **Segment-specific gaps.** The AI personas are most calibrated for Tech and Medical routes to UK and Canada. Less common routes (e.g., Germany Freelance Visa, Portugal D8 Visa) have less test coverage and prompt refinement.

10.3 Quality Control Recommendations

For the next engineer working on this system, the highest-impact accuracy improvements are:

1. **Expand the knowledge base.** For every country and visa category we support, ingest the official policy page via the research endpoint and save it to `migration_documents`. Aim for 50-100 documents minimum.
2. **Restrict Google Search to official domains.** Update `lib/googleSearch.js` to add `site:gov.uk OR site:canada.ca OR site:uscis.gov` filters.
3. **Add a "last verified" badge to lessons.** Show users when the lesson was generated and remind them to check official sources for very recent changes.
4. **Set up a monthly knowledge base refresh.** Re-scrape key policy URLs monthly using the research endpoint and update changed documents.
5. **User feedback loop.** Add a "Was this information helpful/accurate?" thumbs up/down on lessons. Flag low-rated lessons for manual review.

Chapter 11 — The Admin System {#chapter-11}

JapaLearn AI has a team-based admin system for managing the knowledge base and platform content.

11.1 Admin Roles

Role	Permissions
super_admin	Full access, bypasses DB check, can invite/remove admins
admin	Can manage knowledge base documents, view team

The founder's email (`jappalearn@gmail.com`) is hardcoded as a permanent super admin.

11.2 Inviting an Admin

1. Super admin calls `POST /api/admin/team` with `action: "invite"` and the invitee's email
2. A row is created in `admin_invites` with a unique token
3. An invite email is sent (note: email sending mechanism needs verification — check if this is wired to an email provider)
4. Invitee clicks the link → lands on `/admin/join?token=[token]`

5. They create their account
6. Status defaults to `pending` → super admin must approve

11.3 Managing the Knowledge Base

The admin interface allows:

- **Viewing** all documents in the knowledge base (GET `/api/admin/add-document`)
- **Adding** a new document with country, category, source URL (POST `/api/admin/add-document`)
- **Deleting** outdated documents (DELETE `/api/admin/add-document?id=...`)
- **Researching** a URL to auto-generate a document draft (POST `/api/admin/research`)

The research workflow is the recommended way to add new content:

1. Find the official government URL for a visa route
2. Call the research endpoint with that URL
3. Review the AI-generated draft
4. If accurate, save it via `add-document`

11.4 The Research Endpoint in Detail

When given a GOV.UK URL, the endpoint uses the GOV.UK Content API directly:

```
GET https://www.gov.uk/api/content/skilled-worker-visa
```

This returns structured JSON with the official page content — cleaner and more reliable than scraping HTML.

For non-GOV.UK URLs, the endpoint fetches the HTML, strips all tags and scripts, extracts readable text, then sends it to AI for distillation.

The AI is instructed to:

- Extract key requirements (not marketing language)
- Include specific numbers (fees, timelines, thresholds)
- Format as a markdown guide
- Note the source URL and last-updated date

Chapter 12 — Scoring & Assessment Logic {#chapter-12}

12.1 How the Readiness Score Is Calculated

The score (0-100) is calculated by the AI (`SKILLS.READINESS_SCORER`) based on the user's quiz answers. It is not a simple points tally — the AI is instructed to reason like a senior migration analyst.

However, the score maps broadly to these dimensions:

Dimension	Weight	What It Measures
Professional Recognition	20%	Experience years, segment-specific credentials (e.g., MDCN, PCAB), job offer status
Education	20%	Degree level, field relevance, institutional recognition
Language Readiness	20%	IELTS/OET/DELF score or stated plans
Financial Readiness	20%	Savings in NGN relative to destination cost of living and application fees
Process Knowledge	10%	Clarity of destination and visa route, documentation readiness
Age/Profile	10%	Age (most visas favour 25-35 range), family situation impact

Killer criteria (automatic flags regardless of score):

- Savings < ₦5M for non-scholarship routes → Yellow or Red flag immediately
- No language test attempted + low stated proficiency → language gap flagged as critical
- Requesting a destination with no matching visa route for their segment → mismatch warning shown

12.2 Score Bands

Score	Status	Flag
70-100	Strong Candidate	Green
40-69	Moderate Candidate	Yellow
0-39	Early Stage Candidate	Red

12.3 The `calculateScoreBreakdown` Function

In `lib/quizData.js`, there is a function `calculateScoreBreakdown`. This function exists to derive a preliminary breakdown from quiz answers before the AI has run. It is used in the dashboard's Overview tab to show score breakdown without needing a new AI call.

Important rule: Never duplicate or rewrite this scoring logic. Always import and call `calculateScoreBreakdown` from `lib/quizData.js`. If you need to change the scoring, change it in one place only.

12.4 Visa Route Decision Logic

The system recommends a visa route based on destination + segment + experience level + additional answers. Here are example mappings:

UK:

Profile	Recommended Route
Tech (senior, no offer)	Global Talent Visa
Tech (any level, job offer)	Skilled Worker Visa

Profile	Recommended Route
Healthcare Worker	Health and Care Worker Visa
Student	UK Student Visa (Tier 4)
Family member of UK resident	UK Family Visa

Canada:

Profile	Recommended Route
Tech Professional (strong profile)	Express Entry (Federal Skilled Worker)
Student	Study Permit → Post-Graduate Work Permit → PR
Healthcare	Express Entry + Provincial Nominee Program (PNP)

Germany:

Profile	Recommended Route
Freelancer	German Freelance Visa (Freiberufler)
Skilled Worker with job offer	EU Blue Card or Skilled Immigration Act
Student	German Student Visa

These mappings are encoded in both `lib/quizData.js` and in the READINESS_SCORER system prompt.

Chapter 13 — The Curriculum & Learning Engine {#chapter-13}

13.1 Curriculum Architecture

A curriculum has this structure:

```
Curriculum
├── Module 1 (biggest gap first)
│   ├── Lesson 1
│   ├── Lesson 2
│   └── Lesson 3
├── Module 2
│   ├── Lesson 1
│   └── Lesson 2
└── Module N (5-10 total)
```

Each module addresses one specific aspect of the user's migration preparation. The first module always targets the biggest identified gap from the readiness assessment (the "blocker-first" principle).

13.2 Lesson Generation (On-Demand)

Lessons are not pre-generated — they are created on demand when a user first visits a lesson page. This is intentional:

- Reduces upfront AI cost (only generate lessons users actually read)
- Allows context-rich generation at the moment it's needed
- Results are cached after first generation

The lesson writer follows a 9-part structure:

1. **Title** — Clear, specific lesson name
2. **Purpose** — What the user will know after this lesson
3. **Why It Matters** — Connection to their specific visa goal
4. **Prerequisites** — What they should know before starting
5. **Core Explanation** — The main educational content (bulk of the lesson)
6. **Step-by-Step** — Actionable process breakdown
7. **Common Mistakes** — What Nigerian applicants get wrong
8. **Self-Check** — Questions to test understanding
9. **Next Action** — One specific thing to do today

13.3 Module Quizzes

After completing all lessons in a module, users can take a quiz. Quizzes:

- Are generated once and not cached in the DB (regenerated each session — this is a known issue, see Chapter 16)
- Contain 10 questions
- Mix difficulty: 4 easy, 4 medium, 2 hard
- Test content across all lessons in the module
- Show explanations for each answer

13.4 Progress Tracking

The `lesson_progress` table tracks completion per user, per curriculum, per lesson. The dashboard reads this to:

- Show progress bars per module
- Show total % curriculum completion
- Unlock module quizzes (after all lessons in module are complete)
- Show "Recent Activity" on the Overview tab

Chapter 14 — Design System & Styling {#chapter-14}

14.1 Color Palette

Token	Value	Usage
Primary Blue	`#1E4DD7`	Buttons, links, active states
Light Blue	`#3B75FF`	Hover states, accents

Token	Value	Usage
Very Light Blue	`#6094FF`	Backgrounds, badges
Background	`#F7F9FF`	Page background
Surface	`#FFFFFF`	Cards, modals
Heading Text	`#18181B`	H1, H2, strong text
Body Text	`#4D4D56`	Paragraphs, descriptions
Muted Text	`#82858A`	Labels, secondary info
Success Green	`#21C474`	Completion states, green flags
Error Red	`#EF4369`	Error states, red flags
Warning Yellow	`#F5A623`	Yellow flag, caution states

14.2 Typography

Font	Usage	Weights
DM Sans	All headings (H1-H4)	700, 800
Inter	Body text, UI elements, labels	400, 500, 600

Both fonts are loaded via Google Fonts in `_app.js` or via CSS.

14.3 Spacing & Layout

The dashboard uses a fixed sidebar layout:

- Desktop: `260px` fixed left sidebar + remaining width for content
- Mobile: Full-width content, sidebar becomes an overlay triggered by hamburger menu
- Mobile bottom navigation: 5 icons (Home, Learning, Roadmap, Resources, Score)

Standard card padding: `24px` Standard gap between cards: `16px` or `24px`

14.4 Shadows

Custom shadow utilities defined in `tailwind.config.js`:

```
shadow-card     — subtle card elevation
shadow-card-md  — medium card elevation
shadow-card-lg  — prominent card elevation
shadow-btn      — button press effect
shadow-glow     — blue glow for CTA elements
```

14.5 Dark Mode

Dark mode is toggled from the Profile tab in the dashboard. The preference is stored in `localStorage`. The `darkMode` state variable in `dashboard.js` applies a CSS class `dark` to the root element, which triggers Tailwind's dark mode variants.

Dark mode is currently implemented on the dashboard. The landing page and auth pages do not fully support dark mode.

14.6 Animations

Framer Motion is used for:

- Page section entrance animations (slide up + fade in on scroll)
- Staggered list item animations
- Card hover effects
- Sidebar open/close transitions
- Modal enter/exit transitions

Chapter 15 — Environment Variables & Configuration {#chapter-15}

15.1 Environment Variables

All environment variables are stored in `.env.local`. This file is gitignored and must never be committed to the repository.

Variable	Type	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Public	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public	Supabase anon/public key
`SUPABASE_SERVICE_ROLE_KEY`	Secret	Supabase service role key (bypasses RLS — server only)
`GEMINI_API_KEY`	Secret	Google Generative AI API key
`GROQ_API_KEY`	Secret	Groq API key
`OPENAI_API_KEY`	Secret	OpenAI API key
`GOOGLE_SEARCH_API_KEY`	Secret	Google Custom Search API key
`GOOGLE_SEARCH_CX`	Public-ish	Custom Search Engine ID

Important: Variables without `NEXT_PUBLIC_` prefix are only available server-side (in API routes). They are never sent to the browser.

15.2 Next.js Configuration (`next.config.js`)

Currently minimal configuration:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'images.unsplash.com' }
    ]
  }
}
```

When deploying to Vercel, you may need to add the Supabase storage hostname to `remotePatterns` if profile pictures are served from Supabase Storage.

15.3 Running Locally

```
# Install dependencies
npm install

# Start development server
npm run dev
# App runs at http://localhost:3000

# Build for production
npm run build

# Start production server
npm start

# Run linter
npm run lint
```

15.4 Deploying to Vercel

1. Push code to GitHub (main branch)
2. Import repository in Vercel dashboard
3. Add all environment variables from `.env.local` to Vercel project settings
4. Set build command: `npm run build`
5. Set output directory: `.next`
6. Enable automatic deployments from GitHub main branch
7. Every subsequent push to `main` will auto-deploy

Chapter 16 — Known Weaknesses & Technical Debt {#chapter-16}

This chapter is an honest audit of what is incomplete, fragile, or improvable. Every item here was known at the time of writing. This is not a criticism — it is the reality of building an early-stage product at speed.

16.1 Critical Issues

1. Module quiz is not persisted Quizzes are regenerated every session from the AI. If a user takes a quiz, the result is not saved to the database. There is no quiz history or score tracking for module quizzes. The `quiz_results` table only tracks the initial readiness quiz.

Fix: Create a `module_quiz_results` table. Save quiz questions on first generation to a `module_quizzes` table. Save user scores to `module_quiz_results`.

2. Thin knowledge base The RAG knowledge base has only a small number of documents. For most visa routes, the AI is operating from training data alone. Training data may be 1-2 years outdated.

Fix: Systematically ingest official policy pages for all 15 supported destinations across all 8 user categories. Aim for 100+ documents.

3. No error recovery on lesson generation failure If the AI API times out or returns malformed JSON during lesson generation, the lesson page crashes or shows an empty state. There is no retry mechanism or error UI for the user.

Fix: Add try/catch with user-facing error messages and a retry button on the lesson page.

4. No email provider connected for admin invites The admin invite system generates a token but the mechanism for sending the actual invite email needs verification. If there is no email provider (SendGrid, Resend, etc.) configured, invites will silently fail.

Fix: Integrate Resend (recommended — simple API, generous free tier). Add the API key to environment variables.

16.2 Medium Priority Issues

5. Dashboard is one monolithic file (66KB) All dashboard logic lives in `pages/dashboard.js`. This makes it slow to edit, hard to test, and difficult for a new engineer to understand quickly.

Fix: Extract each tab into its own component (`OverviewTab.js`, `LearningTab.js`, etc.). This is a refactor, not a feature — do it in a quiet sprint.

6. No loading states on curriculum generation When a user clicks "Generate Curriculum," the UI does not show meaningful progress. The generation takes 10-30 seconds. Users may click multiple times or navigate away.

Fix: Show a step-by-step loading indicator ("Analysing your profile... Building modules... Ordering lessons...") during curriculum generation.

7. Blog content is static Blog posts are hard-coded in `constants/blogData.js`. Adding new posts requires a code deployment.

Fix: Move blog content to Supabase. Create a `blog_posts` table. Allow admin team to add posts without code changes.

8. Google Search may surface non-official sources The custom search engine is not restricted to official government domains. SEO-heavy immigration agency sites may appear in results and influence AI responses.

Fix: Add domain restrictions to the search engine configuration or append `site:gov.uk OR site:canada.ca` etc. to queries.

9. Newsletter modal has no backend The newsletter modal collects an email address but may not be connected to an email marketing provider. Collected emails may be lost.

Fix: Integrate Mailchimp, ConvertKit, or Brevo. Store emails in a `newsletter_subscribers` table as backup.

10. Shared profile page uses demo data The `pages/shared-profile.js` and `pages/u/[code].js` pages (referral profile sharing) display demo data rather than real user data.

Fix: Wire up the referral profile pages to read actual user quiz results and readiness data by referral code.

16.3 Low Priority / Future Improvements

- No pagination on the blog listing (will matter at 50+ posts)
- No search analytics (cannot see what users search for)
- No A/B testing infrastructure
- Quiz completion rate is unknown (no analytics event tracking)
- Profile picture upload has no size or type validation beyond what the browser enforces
- Dark mode is not applied consistently across all pages

Chapter 17 — Roadmap: From Validation to 100 Users and Beyond {#chapter-17}

17.1 Where We Are Now (Validation Stage)

The product has all core features functioning end-to-end:

- Quiz → Report → Signup → Dashboard → Curriculum → Lesson → Module Quiz

The goal at this stage is to get **100 real users** through the full flow and validate:

1. Do people find the readiness report accurate and useful?
2. Do people start and complete lessons?
3. What segments and destinations generate the most traffic?
4. What questions do users ask that we are not answering?

17.2 Things to Fix Before Scaling

Before inviting more than 100 users, fix these in order:

Priority 1 — Reliability

- Connect email provider for admin invites
- Add error handling on lesson generation failure
- Persist module quiz results

Priority 2 — Accuracy

- Expand knowledge base to 50+ documents
- Restrict Google Search to official domains
- Add "information may be outdated" disclaimer on lessons older than 90 days

Priority 3 — Growth Infrastructure

- Wire up referral profile pages with real data
- Connect newsletter modal to email provider
- Add basic analytics (at minimum: quiz completions, curriculum generations, lesson views)

17.3 Post-Validation Feature Roadmap

Once the first 100 users are validated and product-market fit signals are clear, the following features are prioritised:

Phase 1 — Deepen the Learning Experience

- Community forum / Q&A section (users help each other)
- Expert review option (connect users to verified immigration advisors for paid consultations)
- Video lesson support (embed Loom/YouTube videos into lessons)
- Lesson notes (users can highlight and save notes per lesson)

Phase 2 — Broaden Coverage

- More countries (UAE, Singapore, New Zealand, Australia all under-developed)
- More professional categories (Teaching, Engineering, Accounting in more depth)
- Family reunification pathways in much more detail
- Scholarship database (for the student segment)

Phase 3 — Monetisation

- Freemium model: Quiz + Report free, Curriculum + Lessons behind paywall (₦5,000-₦15,000/month)
- One-off premium reports for specific visa routes
- Partner directory: Verified immigration lawyers per country (referral commission model)
- Group coaching: Cohort-based learning with weekly live sessions

Phase 4 — Intelligence Layer

- Personalised reminders ("Your IELTS results should come back soon — here's what to do next")
- Success tracking (users who reach visa approval self-report outcomes)
- Dynamic score updates (retake mini-quiz monthly to track progress)
- Peer matching (connect users on the same pathway)

17.4 Technical Debt Paydown Plan

Before any Phase 2 features, the engineering team should:

1. Break `dashboard.js` into tab components (1 sprint)
2. Build a proper CMS for blog posts (2 sprints)
3. Add integration tests for all API endpoints (2 sprints)
4. Set up error monitoring (Sentry or similar) (0.5 sprint)
5. Set up proper logging (cannot debug production issues without logs) (0.5 sprint)
6. Add rate limiting to all API endpoints (prevents API cost abuse) (1 sprint)

Appendix A — Full API Reference {#appendix-a}

Method	Path	Auth Required	Purpose
POST	/api/calculate-readiness	No	Score user's readiness + generate report
POST	/api/generate-curriculum	Yes (session)	Build personalised curriculum
POST	/api/generate-lesson	Yes (session)	Generate or retrieve lesson content
POST	/api/generate-quiz	Yes (session)	Generate module quiz
POST	/api/summarise-lesson	Yes (session)	Return 6-point lesson summary
POST	/api/test-ai	No	Test AI connectivity (dev only)
GET	/api/admin/add-document	Admin	List all knowledge base documents
POST	/api/admin/add-document	Admin	Add document to knowledge base
DELETE	/api/admin/add-document	Admin	Remove document from knowledge base
POST	/api/admin/ingest	Admin	Seed initial knowledge base
POST	/api/admin/research	Admin	Scrape and digest a policy URL
GET	/api/admin/team	Admin	List team members and pending invites
POST	/api/admin/team	Admin	Invite/approve/reject/remove admin
GET	/api/admin/join	Public	Validate admin invite token
POST	/api/admin/join	Public	Create admin account from invite

Appendix B — Glossary {#appendix-b}

Term	Definition
Japa	Nigerian slang for leaving the country to relocate abroad
CoS	Certificate of Sponsorship — a code UK employers give skilled workers to apply for a visa
IELTS	International English Language Testing System — English proficiency exam required for many visa routes
OET	Occupational English Test — alternative to IELTS for healthcare professionals
PLAB	Professional and Linguistic Assessments Board — exam required for Nigerian doctors to practise in the UK
MDCN	Medical and Dental Council of Nigeria — the licensing body for Nigerian doctors
NYSC	National Youth Service Corps — compulsory service for Nigerian graduates
NIN	National Identification Number — Nigeria's national ID system
Express Entry	Canada's points-based immigration management system for skilled workers

Term	Definition
PNP	Provincial Nominee Program — Canada's province-level immigration program
PGWP	Post-Graduate Work Permit — allows international students to work in Canada after graduation
RAG	Retrieval-Augmented Generation — AI technique that grounds responses in real documents
Embedding	A numerical vector (list of numbers) that represents text for semantic similarity search
pgvector	A PostgreSQL extension that enables vector storage and similarity search
RLS	Row Level Security — Supabase/PostgreSQL feature that restricts data access per user
NDPR	Nigeria Data Protection Regulation — Nigeria's privacy law
JWT	JSON Web Token — the format Supabase uses for session authentication
CX	Custom Search Engine ID — the identifier for our Google Custom Search configuration
LLM	Large Language Model — the AI technology underlying GPT, Gemini, and Llama
Persona	In this codebase, a SKILL is a persona — a system prompt that defines the AI's role and rules
Readiness Score	A 0-100 number representing how ready a user is to relocate based on their profile
Killer Criterion	A threshold that, if not met, immediately flags a profile as unrealistic regardless of other scores
Segment	The user's specific professional role within their category (e.g., "Medical Doctor" within "Medical")
Module	A themed unit of learning within a curriculum (e.g., "Understanding the UK Visa System")
Lesson	A single educational article within a module, 900-1200 words, grounded in official sources
Curriculum	The full personalised learning plan for a user — typically 5-10 modules, each with 2-5 lessons

---

End of Documentation

This document was written in April 2026 and reflects the state of the codebase at that time. As the product evolves, keep this document updated.